Method and Baseline

A comparison of ResNet-50 and DenseNet-121 for three-class chest X-ray classification

Project	Explainable AI for Chest X-ray Disease Classification
Milestone	Week 2 – Method and Baseline
Dataset	Lung X-Ray Image dataset (3 classes)

Project focus. This project compares two pretrained convolutional neural network models, ResNet-50 and DenseNet-121, for classifying chest X-ray images into Lung Opacity, Normal, and Viral Pneumonia. It also plans to compare the visual explanations produced by Grad-CAM.

1. Computational Method Implemented

The implemented method uses transfer learning with two established image-classification models. Each model starts with pretrained ImageNet weights. Its final classification layer is changed to produce three output scores, one for each chest X-ray class. The models are trained on the same training images and checked against the same validation images.

Before training, the images are organized into training, validation, and testing groups. Images are resized to 224 × 224 pixels and normalized using ImageNet channel statistics. A horizontal flip may be applied to training images as a simple augmentation. The model produces three class scores, and the predicted class is the class with the highest score.

The training process uses cross-entropy loss and the Adam optimizer. The validation loss is used to select the best checkpoint. After training, the saved models are evaluated on the held-out test set. Grad-CAM is included in the implementation to create heat-map visualizations showing image regions that influence a model's prediction. These maps are visual explanations, not proof that a model has found a medically correct region.

2. Source Modules Responsible for the Method

Source file	Purpose
src/config.py	Stores class names, image size, batch size, number of epochs, learning rate, random seed, and worker settings.
src/data/prepare_splits.py	Creates the training, validation, and testing folders from the original class folders.
src/data/dataset.py	Loads images, applies resizing and normalization, and applies the configured training augmentation.

src/models.py	Builds the ResNet-50 and DenseNet-121 models and changes their final layers for three classes.
src/train.py	Runs model training, calculates training and validation loss, and saves the best checkpoint and training history.
src/metrics.py	Calculates classification metrics and the confusion matrix.
src/evaluate.py	Loads a trained checkpoint and evaluates it on the test set; saves metrics and prediction output.
src/explain.py	Generates Grad-CAM visualizations for the trained models.
src/tests/test_metrics.py	Contains unit tests for the metric functions.

3. Baseline, Reference Condition, and Evaluation Criteria

The main comparison is between ResNet-50 and DenseNet-121. Both models use the same three classes, the same prepared data split, the same image size, and the same planned training settings. This provides a common reference condition for comparing their results.

The project does not currently include a separate traditional machine-learning model or a majority-class predictor as an additional baseline. For this milestone, ResNet-50 and DenseNet-121 are the two reference models being compared. The evaluation will use accuracy, macro-averaged precision, macro-averaged recall, macro-averaged F1-score, and a confusion matrix. Macro one-versus-rest ROC-AUC will also be calculated when the test predictions allow it.

The same held-out test set must be used for both models. Accuracy gives the overall proportion of correct predictions. Macro precision, recall, and F1-score give equal weight to each class. The confusion matrix helps show which classes are being confused with one another.

4. Experimental Procedure and Important Parameters

1. Use the three source folders: Lung_Opacity, Normal, and Viral Pneumonia.
2. Create a reproducible image-level split using random seed 42: approximately 70% for training, 15% for validation, and 15% for testing.
3. Load the images and resize them to 224 × 224 pixels. Normalize them using ImageNet mean and standard deviation values.
4. Initialize ResNet-50 with pretrained weights, replace its final classification layer with a three-output layer, and train it.
5. Repeat the process with DenseNet-121 using the same data split and training configuration.
6. Select the best checkpoint using validation loss.
7. Evaluate each saved model on the test set and save the metrics, predictions, and confusion matrix.
8. Generate Grad-CAM images for selected test examples and compare the highlighted regions between the two models.

Parameter	Current setting
Image size	224 × 224 pixels
Batch size	8
Epochs	2 (initial run setting)
Learning rate	0.0001
Optimizer	Adam
Loss function	Cross-entropy
Random seed	42
Execution device	CPU; CUDA is not available in the current environment
Classes	Lung_Opacity, Normal, Viral Pneumonia

5. Instructions for Executing the Experiment

Run the commands from the project root folder in PowerShell. The project dependencies should be installed from requirements.txt.

Install dependencies: python -m pip install -r requirements.txt

Prepare the data split (already completed for this run): python -m src.data.prepare_splits --source "C:\Users\hello\Desktop\Pavani\archive\Lung X-Ray Image\Lung X-Ray Image" --destination data/lung_xray_split --seed 42

Train ResNet-50: python -m src.train --model resnet50

Train DenseNet-121: python -m src.train --model densenet121

Run test-set evaluation: python -m src.evaluate --model resnet50

Evaluate DenseNet-121: python -m src.evaluate --model densenet121

Generate Grad-CAM visualizations: python -m src.explain --model resnet50 (and repeat with densenet121)

Run metric unit tests: python -m unittest discover -s src/tests -v

Note: The exact command-line options should match the current versions of the project scripts. If a command reports an unrecognized argument, check the script's argument parser and README before proceeding. Do not run the data-splitting command again into an existing destination unless the old generated split has first been handled, because old files may remain in the destination.

6. Summary of Initial Experimental Results

The initial data preparation stage completed successfully. The source dataset contained three class folders, and the split script created the training, validation, and testing folders.

Class	Training	Validation	Testing	Total

Lung_Opacity	787	168	170	1,125
Normal	875	187	188	1,250
Viral Pneumonia	770	165	165	1,100
Total	2,432	520	523	3,475

The split contains 2,432 training images, 520 validation images, and 523 testing images, for 3,475 images in total. A separate PowerShell folder-count check confirmed the same counts in all nine class/split folders.

The required Python libraries imported successfully. The environment reported PyTorch 2.15.0+cpu and Torchvision 0.29.2+cpu. CUDA was not available, so the training run is using the CPU.

The ResNet-50 training command was started. The pretrained ResNet-50 weights (97.8 MB) downloaded successfully. During the last process check, the Python process was still active and its reported CPU time had increased from 511 seconds to about 580 seconds. However, no epoch-level training metrics or completed training result had been reported at the time this document was prepared. Therefore, ResNet-50 accuracy, loss, precision, recall, F1-score, and ROC-AUC are not yet available. DenseNet-121 training and final test evaluation are also pending.

These are genuine preliminary results from data preparation and environment checks. They should not be presented as model classification results. The model performance section must be updated after both models complete training and test evaluation.

7. Validation Checks and Tests Performed

Check	Status / evidence
Dataset source folders	Confirmed that Lung_Opacity, Normal, and Viral Pneumonia were present.
Dataset split creation	The split script completed and reported the expected train, validation, and test counts.
Folder-count verification	A PowerShell recursive file count confirmed all nine class/split folders and their counts.
Python dependencies	Imports of torch, torchvision, numpy, PIL, sklearn, and matplotlib completed successfully.
Device check	PyTorch reported CPU execution and CUDA unavailable.
Model training status	ResNet-50 pretrained weights downloaded; the training process was still active at the last check, but completion was not confirmed.
Unit tests	A test file is present at src/tests/test_metrics.py. A successful test run has not yet been confirmed.

8. Significant Methodological Problems and Unresolved Questions

- **CPU training time:** The current computer is using CPU-only PyTorch. Training may take longer than it would on a supported GPU. Training completion and runtime still need to be recorded.
- **Short initial training:** The current configuration uses two epochs. This is suitable for an initial pipeline check, but it may not be enough for a reliable final comparison. Further training and tuning may be needed.
- **Image-level splitting:** The current split is random at the image level. The available folder information does not show patient identifiers. If multiple images from the same patient exist across splits, the evaluation may be overly optimistic. A patient-level split would be better if patient IDs are available.
- **Dataset provenance:** The dataset source is the Kaggle Lung X-Ray Image dataset. The dataset description, original collection process, patient-level grouping, and license should be checked and documented before final submission.
- **Class imbalance:** The class totals are not identical. Macro-averaged metrics are included so that each class has equal weight in the summary.
- **Grad-CAM interpretation:** Grad-CAM heat maps show areas that affect model output, but they do not prove that the model is using medically meaningful features. The images should be reviewed carefully and described as explanations, not clinical evidence.
- **Results still pending:** Both models must complete training and test evaluation before a performance comparison can be made. No model accuracy or other performance values are claimed in this report.

Current Week 2 Status

The data preparation and environment checks are complete. The source code includes the modules for training, evaluation, metrics, and Grad-CAM. ResNet-50 training has been started, but its completion and model metrics have not yet been confirmed. DenseNet-121 training, test evaluation, Grad-CAM inspection, and the metric unit-test result remain to be completed or verified.